

Step 1: Check GCC

```
gcc --version
```

```
g++ --version
```

If not installed:

```
sudo apt update
```

```
sudo apt install build-essential
```

Step 2: Save your code

```
nano bfs_dfs.cpp
```

```
CTRL + O → Enter → CTRL + X
```

Step 3: Compile with OpenMP

```
g++ -fopenmp bfs_dfs.cpp -o hpc
```

Step 4: Run

```
./hpc
```

code :

```
#include <iostream>
```

```
#include <vector>
```

```
#include <climits>
```

```
#include <omp.h>
```

```
#include <chrono>
```

```
#include <random>
```

```
#include <cmath>
```

```
using namespace std;
```

```
// ————— SEQUENTIAL —————
```

```
int seq_min(const vector<int>& arr) {  
    int mn = INT_MAX;  
    for (int x : arr) mn = min(mn, x);  
    return mn;  
}
```

```
int seq_max(const vector<int>& arr) {  
    int mx = INT_MIN;  
    for (int x : arr) mx = max(mx, x);  
    return mx;  
}
```

```
long long seq_sum(const vector<int>& arr) {  
    long long s = 0;  
    for (int x : arr) s += x;  
    return s;  
}
```

```
double seq_avg(const vector<int>& arr) {  
    return (double)seq_sum(arr) / arr.size();  
}
```

```
// ————— PARALLEL (REDUCTION) —————
```

```
int par_min(const vector<int>& arr) {  
    int mn = INT_MAX;  
  
    #pragma omp parallel for reduction(min:mn)  
    for (int i = 0; i < arr.size(); i++)
```

```

        mn = min(mn, arr[i]);

    return mn;
}

int par_max(const vector<int>& arr) {
    int mx = INT_MIN;

    #pragma omp parallel for reduction(max:mx)
    for (int i = 0; i < arr.size(); i++)
        mx = max(mx, arr[i]);

    return mx;
}

long long par_sum(const vector<int>& arr) {
    long long s = 0;

    #pragma omp parallel for reduction(+:s)
    for (int i = 0; i < arr.size(); i++)
        s += arr[i];

    return s;
}

double par_avg(const vector<int>& arr) {
    return (double)par_sum(arr) / arr.size();
}

// ————— TIMER —————

```

```

template<typename Func>
double timelt(Func f) {
    auto start = chrono::high_resolution_clock::now();
    f();
    auto end = chrono::high_resolution_clock::now();
    return chrono::duration<double, milli>(end - start).count();
}

//————— MAIN —————

int main() {

    const int N = 10000000; // safer for lab PCs

    vector<int> arr(N);

    mt19937 rng(99);
    uniform_int_distribution<int> dist(1, 1000000);

    for (auto &x : arr) x = dist(rng);

    cout << "=====\n";
    cout << " Parallel Reduction using OpenMP\n";
    cout << "=====\n";

    cout << "Array size: " << N << endl;
    cout << "Threads: " << omp_get_max_threads() << endl;

    // MIN

    int s_min, p_min;
    double t_s = timelt([&]{ s_min = seq_min(arr); });

```

```

double t_p = timelt([&]{ p_min = par_min(arr); });

cout << "\n--- MIN ---\n";
cout << "Seq: " << s_min << " Time: " << t_s << " ms\n";
cout << "Par: " << p_min << " Time: " << t_p << " ms\n";
cout << "Speedup: " << t_s/t_p << "x\n";

// MAX
int s_max, p_max;
t_s = timelt([&]{ s_max = seq_max(arr); });
t_p = timelt([&]{ p_max = par_max(arr); });

cout << "\n--- MAX ---\n";
cout << "Seq: " << s_max << " Time: " << t_s << " ms\n";
cout << "Par: " << p_max << " Time: " << t_p << " ms\n";
cout << "Speedup: " << t_s/t_p << "x\n";

// SUM
long long s_sum, p_sum;
t_s = timelt([&]{ s_sum = seq_sum(arr); });
t_p = timelt([&]{ p_sum = par_sum(arr); });

cout << "\n--- SUM ---\n";
cout << "Seq: " << s_sum << " Time: " << t_s << " ms\n";
cout << "Par: " << p_sum << " Time: " << t_p << " ms\n";
cout << "Speedup: " << t_s/t_p << "x\n";

// AVG
double s_avg, p_avg;
t_s = timelt([&]{ s_avg = seq_avg(arr); });
t_p = timelt([&]{ p_avg = par_avg(arr); });

```

```

cout << "\n--- AVG ---\n";

cout << "Seq: " << s_avg << " Time: " << t_s << " ms\n";
cout << "Par: " << p_avg << " Time: " << t_p << " ms\n";
cout << "Speedup: " << t_s/t_p << "x\n";

cout << "\nCheck Avg Match: "
    << (fabs(s_avg - p_avg) < 1e-6 ? "YES" : "NO") << endl;

cout << "=====\n";

return 0;
}

```

◆ Step 1: Install WSL

Open PowerShell (Admin):

wsl --install

Restart PC

◆ Step 2: Open Ubuntu terminal

Search → "Ubuntu"

◆ Step 3: Install GCC + OpenMP

sudo apt update

sudo apt install build-essential

◆ Step 4: Run your HPC code

```
nano program.cpp  
g++ -fopenmp program.cpp -o run  
./run
```